

Project Roadmap

Security of ESP32-Based IoT Systems, Distributed Monitoring, and Federated Learning

General Objective

The project aims to design and experiment with an IoT infrastructure composed of multiple **ESP32** devices, programmed in **C/C++**, interconnected and monitored through a system capable of analyzing both device behavior and network communications.

The activity focuses on studying how to detect anomalies, possible attacks, or suspicious behaviors within an IoT network, identifying the involved devices and analyzing the possible causes of the detected events.

A possible evolution of the project concerns the introduction of a **Federated Learning** approach, in which devices or monitoring nodes collaboratively train detection models without directly sharing the raw data generated by the IoT network.

Phase 1 — Preliminary Study and Acquisition of Prerequisites

Objectives

Understand the technological and theoretical foundations required to address the project.

Activities

The student should study the following topics:

- architecture and main characteristics of ESP32 devices;
- embedded programming in C/C++ for microcontrollers;
- Wi-Fi communication between IoT devices;
- relevant communication protocols, such as TCP/IP, UDP, HTTP, and MQTT;
- security principles in IoT networks;
- basic concepts of Network Intrusion Detection Systems;
- main types of attacks in IoT networks, such as scanning, flooding, spoofing, denial of service, traffic anomalies, and abnormal communication patterns;
- introduction to Machine Learning for anomaly detection;
- fundamental concepts of Federated Learning.

Expected Outcome

At the end of this phase, the student should produce a short technical summary of the studied concepts, highlighting which aspects will be useful for the development of the project.

Phase 2 — Design of the IoT Architecture

Objectives

Define a small experimental IoT network based on ESP32 devices.

Activities

The network may consist of multiple ESP32 nodes, each with a specific role, for example:

- sensor node, responsible for generating or collecting data;
- actuator or receiver node;
- gateway node, responsible for collecting information or forwarding messages;
- monitoring node, possibly external to the ESP32 devices, such as a PC or Raspberry Pi.

The student should design:

- the network topology;
- the communication model between the devices;
- the format of the exchanged messages;
- the parameters to be observed for monitoring;
- possible normal and anomalous scenarios.

Expected Outcome

A diagram of the IoT network architecture, a description of the device roles, and a definition of the communication flows.

Phase 3 — Implementation of the ESP32 Network

Objectives

Develop a first working version of the IoT network.

Activities

The student should:

- configure the ESP32 devices;
- implement communication between the nodes;
- generate regular traffic among the devices;
- collect logs related to sent and received messages, errors, delays, and device status;
- verify the stability of the communication.

Depending on the selected level of complexity, communication may be based on HTTP, UDP, TCP, or MQTT.

Expected Outcome

A working ESP32 network capable of exchanging data in a controlled and reproducible way.

Phase 4 — Traffic Monitoring and Data Collection

Objectives

Build a system for observing the behavior of the network.

Activities

The system should collect useful information to distinguish normal behavior from anomalous behavior. Possible data to collect include:

- number of sent and received packets;
- message frequency;
- source node address or identifier;
- destination node address or identifier;
- message size;
- time between consecutive messages;
- transmission errors;
- delays;
- disconnections;
- sudden variations in traffic.

Monitoring may be implemented:

- directly on the ESP32 devices through local logs;
- on a gateway;
- on an external node, such as a PC or Raspberry Pi;
- through network traffic analysis tools.

Expected Outcome

An initial dataset containing examples of normal traffic and, subsequently, anomalous or simulated traffic.

Phase 5 — Definition of Attack or Anomaly Scenarios

Objectives

Create controlled scenarios to test the monitoring system.

Activities

The student should design and simulate some anomalous scenarios, such as:

- sending an excessive number of messages from one node;
- behavior of a compromised device;
- unauthorized communication attempts;
- messages with an unexpected format;
- anomalous traffic directed toward a specific node;
- frequent disconnections or reconnections;
- abnormal communication delays;
- a possible lightweight and controlled denial-of-service scenario.

The goal is not to implement complex attacks, but to build experimental scenarios useful for evaluating the system's ability to detect suspicious behavior.

Expected Outcome

A documented set of normal and anomalous scenarios, together with the corresponding collected data.

Phase 6 — Development of a First Detection System

Objectives

Implement an initial Network Intrusion Detection or anomaly detection logic.

Activities

In a first version, the system may rely on simple rule-based mechanisms, such as:

- thresholds on the number of messages per unit of time;
- detection of messages coming from unexpected nodes;
- identification of packets with incorrect formats;
- detection of anomalous communication frequencies;
- identification of suspicious patterns in the logs.

Subsequently, a Machine Learning-based approach may be introduced, for example:

- binary classification: normal traffic / anomalous traffic;

- multi-class classification: normal, flooding, invalid message, suspicious node, etc.;
- unsupervised anomaly detection, for example using Isolation Forest, One-Class SVM, or autoencoders.

Expected Outcome

A first prototype of a detection system capable of reporting suspicious behaviors in the IoT network.

Phase 7 — Identification of the Potentially Compromised Node

Objectives

Associate the detected anomaly with one or more devices in the network.

Activities

The system should try to answer questions such as:

- which device generated the anomalous traffic?
- which node was affected?
- does the anomaly concern a single node or the entire network?
- is the suspicious behavior isolated or recurrent?
- which traffic features caused the alert?

This phase may also include a simple explainability component, useful for justifying the alert generated by the system.

Expected Outcome

An analysis module capable of indicating the suspicious device and providing a concise explanation of the detected anomaly.

Phase 8 — Introduction of Federated Learning

Objectives

Study and design a federated extension of the detection system.

Motivation

In a real IoT scenario, the data produced by devices may be distributed, sensitive, or difficult to centralize. A traditional approach would require collecting all data on a single central server to train a detection model.

Federated Learning instead enables collaborative model training while keeping data locally stored on the nodes or gateways.

Activities

The student should analyze how Federated Learning can be applied to the project. A possible architecture may include:

- multiple local nodes or gateways collecting data from the IoT network;
- a local model trained on each node;
- a central server aggregating the parameters of the local models;
- an updated global model obtained without transferring raw data.

As a first step, Federated Learning may be simulated on a PC by using the datasets collected from the ESP32 devices and artificially splitting them among multiple clients. Subsequently, if feasible, a more realistic deployment on gateways or edge devices may be considered.

Possible technologies include:

- Python for the Machine Learning part;
- Flower as a Federated Learning framework;
- Scikit-learn or PyTorch for the models;
- ESP32 for generating IoT data;
- PC or Raspberry Pi as edge/gateway nodes.

Expected Outcome

A prototype or simulation of a federated system for anomaly detection in IoT networks.

Phase 9 — Experimental Evaluation

Objectives

Evaluate the implemented system in terms of accuracy, robustness, and practical usefulness.

Possible Metrics

For the detection system:

- accuracy;
- precision;
- recall;
- F1-score;
- false positive rate;
- false negative rate;

- detection latency.

For the federated system:

- comparison between centralized and federated models;
- global accuracy;
- performance of individual clients;
- impact of non-uniform data distribution;
- number of federated rounds required;
- amount of data not directly shared;
- computational cost.

Expected Outcome

An evaluation table or report comparing rule-based approaches, centralized Machine Learning, and, if implemented, Federated Learning.

Phase 10 — Final Documentation and Possible Developments

Objectives

Produce complete and reproducible technical documentation.

Activities

The student should document:

- system architecture;
- ESP32 configuration;
- developed code;
- test scenarios;
- collected dataset;
- detection methodology;
- possible federated approach;
- experimental results;
- limitations of the system;
- possible future developments.

Possible Extensions

The project may be extended in several directions:

- use of more ESP32 devices;
- integration with real sensors;
- use of MQTT and a dedicated broker;
- deployment on Raspberry Pi as an edge gateway;

- more advanced anomaly detection models;
- Federated Learning with non-IID data;
- detection of more sophisticated attacks;
- dashboard for real-time monitoring;
- integration with automatic alerts;
- energy analysis of IoT devices.

Expected Outcome

A final technical report, the code repository, and a demonstration of the developed system.